

```
1 =====
2 NETWORK MARVELS – SOFTWARE ENGINEER, THANE
3 ADDITIONAL QUESTIONS NOT IN YOUR PREP PDFS
4 For: Samaranjan Manjhi | Interview: 16 June 2026, 2 PM
5 =====
6 These are questions NOT already covered in your two prep documents.
7 Grouped by round type. Study in order – highest probability first.
8 =====
9
10
11 =====
12 ROUND 1 – APTITUDE / WRITTEN (30-40 min, likely first)
13 =====
14
15 --- OUTPUT-BASED C QUESTIONS (not in your PDFs) ---
16
17 Q1. What is the output?
18     int a = 10, b = 5;
19     printf("%d", a/b + a%b);
20     ANSWER: 2 + 0 = 2
21
22 Q2. What is the output?
23     int x = 7;
24     x = x++ + ++x;
25     printf("%d", x);
26     ANSWER: UNDEFINED BEHAVIOR – say "UB, multiple side effects on x
27     without sequence point." This impresses interviewers.
28
29 Q3. What is the output?
30     char arr[] = "Hello";
31     printf("%d %d", strlen(arr), sizeof(arr));
32     ANSWER: 5 6 (strlen ignores '\0', sizeof includes it)
33
34 Q4. What is the output?
35     int arr[3] = {1, 2, 3};
36     int *p = arr;
37     printf("%d", *(p++));
38     printf("%d", *p);
39     ANSWER: 1 2 (post-increment: print arr[0] first, then pointer
40     moves, print arr[1])
41
42 Q5. What is the output?
43     int x = 5;
44     int y = x > 3 ? (x > 4 ? 10 : 20) : 30;
45     printf("%d", y);
46     ANSWER: 10 (x>3 true, x>4 true → 10)
47
48 Q6. What is the output?
49     int i = 0;
50     while(i++ < 3)
51         printf("%d ", i);
```

ANSWER: 1 2 3 (i incremented AFTER comparison, printed after)

Q7. What is the output?

```
printf("%d", !0 + !0 - !1);
```

ANSWER: $1 + 1 - 0 = 2$

Q8. What is the size?

```
struct S { int a; char b; int c; char d; };
```

```
sizeof(S) = ?
```

ANSWER: 16 bytes

Layout: $\text{int}(4) + \text{char}(1) + \text{pad}(3) + \text{int}(4) + \text{char}(1) + \text{pad}(3) = 16$

Q9. What is the output?

```
int arr[] = {10, 20, 30, 40, 50};
```

```
printf("%d", arr[4] - arr[1]);
```

ANSWER: $50 - 20 = 30$

Q10. What is the output?

```
int a = 0;
```

```
a = a || (a = 5);
```

```
printf("%d", a);
```

ANSWER: 1 (short-circuit: $a=0$ is false, evaluates right side $a=5$, result of $||$ is 1, so $a=1$ NOT 5 – common trap!)

--- APTITUDE MATH (expect 5-10 questions, 1-2 min each) ---

Q. A can do a work in 12 days, B in 18 days. They work together for 4 days, then A leaves. How many more days for B to finish?

ANSWER: Together rate = $1/12 + 1/18 = 5/36$. In 4 days: $20/36 = 5/9$.

Remaining: $4/9$. B alone: $(4/9) \times 18 = 8$ days.

Q. A train 300m long passes a platform 200m long in 25 seconds.

What is the speed of the train?

ANSWER: Distance = 500m, Time=25s. Speed = $500/25 = 20$ m/s = 72 km/h

Q. What is the compound interest on Rs 8000 at 10% for 2 years?

ANSWER: $CI = 8000(1+0.1)^2 - 8000 = 8000 \times 1.21 - 8000 = 1680$

Q. In how many ways can 4 boys and 3 girls sit in a row such that no two girls sit adjacent?

ANSWER: Arrange 4 boys: $4! = 24$. Gaps for girls: _B_B_B_B_ = 5 gaps.

Choose 3: $5P3 = 60$. Total = $24 \times 60 = 1440$.

Q. A number when divided by 6 gives remainder 3, divided by 9 gives remainder 3. What is the smallest such number?

ANSWER: Number = $\text{LCM}(6,9) + 3 = 18 + 3 = 21$. Verify: $21\%6=3$, $21\%9=3$. ✓

=====

ROUND 2 – TECHNICAL (C/C++ Theory + OS Internals)

=====

These questions are based on your resume projects – they WILL dig deep.

--- FANOTIFY / MW GUARDIAN DEEP DIVES ---

Q. What is the difference between FAN_OPEN_PERM and FAN_CLOSE_WRITE?

ANSWER: FAN_OPEN_PERM is a permission event – it BLOCKS the operation waiting for ALLOW/DENY response. FAN_CLOSE_WRITE is a notify event – it fires AFTER the file is written and closed, no blocking. In MW Guardian I use FAN_OPEN_PERM to intercept exec() BEFORE the binary runs. FAN_CLOSE_WRITE would be used for post-write auditing.

Q. What happens if your fanotify verifier process crashes mid-verification?

ANSWER: The frozen binary remains blocked only until fanotify timeout. In practice: I close the fanotify fd in the verifier crash handler – kernel auto-sends ALLOW on fd close. Real fix: systemd watchdog restarts the verifier. Also, the kernel has its own timeout for permission events.

Q. How do you handle race conditions in fanotify event processing?

ANSWER: fanotify fd is read by a single event loop thread. Responses (ALLOW/DENY) are written back using the event's fd. Since each event has a unique file descriptor for the response, there is no race on the response path. The signature verification itself is stateless – reads from the binary and the key store only.

Q. What if two different processes try to execute the same binary simultaneously – how does your guardian handle it?

ANSWER: fanotify generates one permission event per exec() call, regardless of how many processes execute the same binary. Each event is handled independently. If both pass signature check, both get ALLOW. No shared state issue since verification is read-only.

Q. How does Ed25519 signature verification work at a high level?

ANSWER: Signer: takes private key + message → produces 64-byte signature. Verifier: takes public key + message + signature → returns valid/invalid. Security: based on elliptic curve discrete log problem. libsodium's crypto_sign_verify_detached() handles this in a single call. The binary content is the message, signature stored in extended attributes or a side-car file.

Q. Where do you store the signature for each binary?

ANSWER: Two options I evaluated – extended file attributes (xattr) or a separate .sig sidecar file. I chose a central SQLite database keyed by binary path + inode number, because xattr support varies across filesystems and sidecar files create deployment complexity.

--- FTRACE / RANSOMWARE MODULE DEEP DIVES ---

Q. What is the difference between ftrace and eBPF for syscall hooking?

ANSWER: ftrace is older, simpler, modifies kernel function trampolines directly. Requires a kernel module. eBPF is the modern approach – loads verified bytecode into kernel, sandboxed, safer, can be updated

without reloading a module. For my ransomware module I used ftrace because it gives direct syscall control (SIGKILL from handler), whereas eBPF programs cannot block or kill processes as easily (needs helper programs). In production eBPF would be preferred for monitoring.

Q. How does your module survive a rmmod attempt?

ANSWER: I remove the module's list_head from the kernel module list (kallsyms lookup + list_del). After this, the module is invisible to lsmod and cannot be removed via rmmod since the kernel cannot find it in its module list. The sysfs interface requires a password – the rmmod call itself fails because the module entry is gone from the list. This is the same technique rootkits use, applied defensively.

Q. What is the risk of your ransomware detection causing false positives?

ANSWER: Real risk – backup software, batch file processors, and build systems can trigger bulk-rename thresholds. My configurable threshold at insmod time helps: set it high for servers running backups. Better production approach: combine rename threshold with extension pattern matching AND entropy check on file contents (encrypted files have high entropy). I handle the first two; entropy check is a planned improvement.

--- UNIX SOCKET / IPC DEEP DIVES ---

Q. Why Unix domain sockets over TCP localhost for your scanner IPC?

ANSWER: Unix domain sockets stay entirely in kernel – no TCP stack, no IP routing, no port allocation, no loopback overhead. Faster for local IPC. Also: file permissions control access (chmod on socket file), unlike TCP where any local process can connect to a port. For a security daemon, access control on the socket file is important.

Q. How do you handle multiple simultaneous scan clients on your socket server?

ANSWER: I used epoll in edge-triggered mode. One listening socket, multiple client connections tracked in epoll. Each client sends a file path, gets a scan result back. The PCRE2 engine and file handlers are shared (read-only) – no locking needed. SQLite for logging uses WAL mode to allow concurrent writes.

Q. What is the format of messages on your Unix socket?

ANSWER: I use a simple length-prefixed protocol: 4-byte uint32 message length, then the UTF-8 file path. Response: 4-byte length + JSON result. Length prefix is critical – otherwise you can't tell where one message ends on a stream socket.

--- C++ THEORY (NOT in your PDFs) ---

Q. What is the difference between static_cast, dynamic_cast, reinterpret_cast, and const_cast?

ANSWER:

static_cast: compile-time cast, no runtime check. Use for numeric conversions, upcasting. Fast.

dynamic_cast: runtime check via RTTI for downcasting pointers/refs.
Returns nullptr (or throws) on failure. Use when you're not sure.
reinterpret_cast: raw bit reinterpretation. No check. Use for
pointer↔integer, hardware registers. Dangerous.
const_cast: add/remove const only. Use to call non-const API with
const data (when you know it won't modify).
YOUR LINK: "In my kernel module I use reinterpret_cast when casting
between kernel struct pointers – same idea as C-style cast but explicit."

Q. What is CRTP (Curiously Recurring Template Pattern)?

ANSWER: A pattern where a class inherits from a template instantiated
with itself:
template<typename Derived>
class Base { void interface() { static_cast<Derived*>(this)->impl(); } };
class Child : public Base<Child> { void impl() { ... } };
Achieves static polymorphism – virtual dispatch at compile time.
Zero runtime overhead. Used in embedded/performance-critical code.

Q. What is the difference between placement new and regular new?

ANSWER: Regular new: allocates memory + constructs object.
Placement new: constructs object IN pre-allocated memory.
char buf[sizeof(MyClass)];
MyClass* p = new(buf) MyClass(); // no allocation, just construction
Use when: memory pools, shared memory, custom allocators.
Must manually call destructor: p->~MyClass();
YOUR LINK: "Relevant to kernel-space where you can't use the heap
allocator and must use pre-allocated buffers."

Q. What is std::move vs std::forward?

ANSWER: std::move: unconditionally casts to rvalue reference.
"I want to move this, treat it as temporary."
std::forward: conditionally casts – preserves the value category
of the original argument (perfect forwarding in templates).
template<typename T>
void wrapper(T&& arg) { target(std::forward<T>(arg)); }
// If arg was lvalue → forward as lvalue. rvalue → forward as rvalue.

Q. What is the difference between emplace_back and push_back?

ANSWER: push_back: constructs object outside, then copies/moves it in.
emplace_back: constructs object IN-PLACE inside the vector.
No temporary created. More efficient for non-trivial objects.
vec.push_back(MyObj(1, 2)); // construct temp, then move
vec.emplace_back(1, 2); // construct directly in vec

Q. What is the difference between std::array and C-style array?

ANSWER: C-style: decays to pointer when passed to function (loses size).
std::array: knows its size, doesn't decay, range-for compatible,
can be returned from function, STL algorithms work on it.
std::array<int, 5> a = {1,2,3,4,5};
a.size(); // 5 – always available

Q. What is a memory pool and why would you use it?

ANSWER: Pre-allocate a large block of memory, then sub-allocate from it with $O(1)$ alloc/free. Benefits: avoid malloc overhead (syscall cost), avoid fragmentation, predictable performance.

YOUR LINK: "In my fanotify event loop, if I were allocating a struct per event under high load, a pool allocator would avoid repeated malloc/free calls. Currently I use a fixed-size ring buffer instead."

Q. What is the difference between a mutex and a spinlock in the kernel?

ANSWER: In kernel context: spinlock = busy-wait, cannot sleep, must be used in interrupt handlers (interrupt context cannot sleep).

Mutex = can sleep (puts thread to sleep until unlocked), only usable in process context (not in interrupt handlers).

YOUR LINK: "In my Linux kernel module I used spinlock_t for the rename counter because the ftrace callback fires in an interrupt-like context – sleeping is not allowed there."

Q. What is the difference between synchronous and asynchronous I/O?

ANSWER: Synchronous: caller blocks until I/O completes.

Asynchronous: caller submits I/O request and continues. OS notifies when complete (epoll/IOCP/io_uring).

YOUR LINK: "My Unix socket server uses epoll (async) – event loop handles hundreds of clients without blocking. Contrast with my fanotify loop which is synchronous – I read an event, verify, respond, then read next."

--- OS INTERNALS (not in your PDFs) ---

Q. What is the difference between a hard link and a symbolic link?

ANSWER: Hard link: another directory entry pointing to the same inode. Same file, different name. Cannot cross filesystems. File data lives as long as any hard link exists.

Symbolic link: a file that contains a path to another file. Can cross filesystems. Target may not exist (dangling symlink). Deleting target breaks the symlink.

YOUR LINK: "In FIM, I watch inodes not paths – so hard links to a watched file are automatically covered. Symlinks to watched paths need explicit handling."

Q. What is copy-on-write (COW) in the context of fork()?

ANSWER: After fork(), parent and child share the same physical memory pages (mapped read-only for both). When either writes to a page, the kernel creates a private copy for that process. Avoids copying all memory at fork time – only pages that are actually written are copied. This is why fork() is fast even for large processes.

Q. What is mmap and how is it different from read()?

ANSWER: read(): copies data from kernel buffer cache to user buffer. mmap(): maps file directly into virtual address space. Access it like memory – no explicit read() call. Page fault on first access loads the page. Fast for large files or random access patterns.

YOUR LINK: "When my binary verifier needs to hash the binary content,
I use mmap instead of read() – maps the file, passes pointer to
libsodium's hash function. No extra copy."

Q. What is an inode?

ANSWER: Inode = metadata structure for a file: permissions, owner,
timestamps, file size, block pointers (location of data on disk).
Does NOT contain the filename – the directory entry maps name-inode.
YOUR LINK: "My FIM module stores the inode number in SQLite as the
primary key – more reliable than path for detecting renames."

Q. What is the difference between fork() and vfork()?

ANSWER: fork(): creates child with copy-on-write copy of parent.
Child can modify its own memory.
vfork(): child runs in parent's address space until exec() or exit().
No copy at all. Faster but dangerous – child cannot safely use
parent's stack or call most functions. Used historically before COW.
Modern code: always use fork(). Or posix_spawn() for new process.

=====

ROUND 3 – DSA CODING (NOT in your PDFs)

=====

These are commonly asked at mid-level C++ interviews and NOT covered
in your existing prep docs. All in C++.

--- LINKED LIST ---

Q. Merge two sorted linked lists.

```
ListNode* mergeTwoLists(ListNode* l1, ListNode* l2) {  
    ListNode dummy(0);  
    ListNode* tail = &dummy;  
    while(l1 && l2) {  
        if(l1->val <= l2->val) { tail->next=l1; l1=l1->next; }  
        else { tail->next=l2; l2=l2->next; }  
        tail = tail->next;  
    }  
    tail->next = l1 ? l1 : l2;  
    return dummy.next;  
}
```

Time: $O(n+m)$, Space: $O(1)$

Q. Remove Nth node from end of linked list.

```
ListNode* removeNthFromEnd(ListNode* head, int n) {  
    ListNode dummy(0);  
    dummy.next = head;  
    ListNode *fast=&dummy, *slow=&dummy;  
    for(int i=0; i<=n; i++) fast=fast->next; // fast is n+1 ahead  
    while(fast) { slow=slow->next; fast=fast->next; }  
    slow->next = slow->next->next;  
    return dummy.next;  
}
```

```
    }  
    Time: O(n), Space: O(1) – one pass with two pointers
```

Q. Check if linked list is a palindrome.

```
bool isPalindrome(ListNode* head) {  
    // Step 1: find middle  
    ListNode *slow=head, *fast=head;  
    while(fast && fast->next) { slow=slow->next; fast=fast->next->next; }  
    // Step 2: reverse second half  
    ListNode *prev=nullptr, *curr=slow;  
    while(curr) { ListNode* nxt=curr->next; curr->next=prev; prev=curr; curr=nxt; }  
    // Step 3: compare  
    ListNode *left=head, *right=prev;  
    while(right) { if(left->val!=right->val) return false; left=left->next; right=right->next; }  
    return true;  
}  
Time: O(n), Space: O(1)
```

--- STRINGS (NOT in your PDFs) ---

Q. Longest palindromic substring.

```
string longestPalindrome(string s) {  
    int start=0, maxLen=1, n=s.size();  
    auto expand = [&](int l, int r) {  
        while(l>=0 && r<n && s[l]==s[r]) { l--; r++; }  
        if(r-l-1 > maxLen) { maxLen=r-l-1; start=l+1; }  
    };  
    for(int i=0; i<n; i++) { expand(i,i); expand(i,i+1); }  
    return s.substr(start, maxLen);  
}  
Time: O(n2), Space: O(1)
```

Q. Group anagrams together.

```
vector<vector<string>> groupAnagrams(vector<string>& strs) {  
    unordered_map<string, vector<string>> mp;  
    for(string& s : strs) {  
        string key = s;  
        sort(key.begin(), key.end());  
        mp[key].push_back(s);  
    }  
    vector<vector<string>> res;  
    for(auto& p : mp) res.push_back(p.second);  
    return res;  
}  
Time: O(n * k log k) where k = max string length
```

Q. Valid palindrome (ignore non-alphanumeric).

```
bool isPalindrome(string s) {  
    int l=0, r=s.size()-1;  
    while(l<r) {  
        while(l<r && !isalnum(s[l])) l++;
```



```
        while(l<r && !isalnum(s[r])) r--;
```

```
        if(tolower(s[l]) != tolower(s[r])) return false;
```

```
        l++; r--;
```

```
    }
```

```
    return true;
```

```
}
```

Q. Find all permutations of a string.

```
void permute(string& s, int start, vector<string>& res) {
```

```
    if(start == s.size()) { res.push_back(s); return; }
```

```
    for(int i=start; i<s.size(); i++) {
```

```
        swap(s[start], s[i]);
```

```
        permute(s, start+1, res);
```

```
        swap(s[start], s[i]); // backtrack
```

```
    }
```

```
}
```

```
Time: O(n * n!), Space: O(n)
```

Q. Count and say sequence (common in aptitude coding).

```
string countAndSay(int n) {
```

```
    string res = "1";
```

```
    for(int i=1; i<n; i++) {
```

```
        string next = "";
```

```
        int j=0;
```

```
        while(j<res.size()) {
```

```
            char c = res[j]; int cnt=0;
```

```
            while(j<res.size() && res[j]==c) { j++; cnt++; }
```

```
            next += to_string(cnt) + c;
```

```
        }
```

```
        res = next;
```

```
    }
```

```
    return res;
```

```
}
```

```
// 1 → "1", 2 → "11", 3 → "21", 4 → "1211", 5 → "111221"
```

--- STACK / QUEUE (NOT in your PDFs) ---

Q. Implement a stack that supports getMin() in O(1).

```
class MinStack {
```

```
    stack<int> st, minSt;
```

```
public:
```

```
    void push(int val) {
```

```
        st.push(val);
```

```
        if(minSt.empty() || val <= minSt.top()) minSt.push(val);
```

```
    }
```

```
    void pop() {
```

```
        if(st.top() == minSt.top()) minSt.pop();
```

```
        st.pop();
```

```
    }
```

```
    int top() { return st.top(); }
```

```
    int getMin() { return minSt.top(); }
```

```
};
```

Q. Next greater element for each element in array.

```
vector<int> nextGreater(vector<int>& arr) {  
    int n=arr.size();  
    vector<int> res(n, -1);  
    stack<int> st; // stores indices  
    for(int i=0; i<n; i++) {  
        while(!st.empty() && arr[i] > arr[st.top()]) {  
            res[st.top()] = arr[i];  
            st.pop();  
        }  
        st.push(i);  
    }  
    return res;  
}  
// [4,5,2,10] → [5,10,10,-1]  
Time: O(n), Space: O(n)
```

Q. Implement queue using two stacks.

```
class MyQueue {  
    stack<int> in, out;  
    void transfer() { while(!in.empty()) { out.push(in.top()); in.pop(); } }  
public:  
    void push(int x) { in.push(x); }  
    int pop() { if(out.empty()) transfer(); int v=out.top(); out.pop(); return v; }  
    int peek() { if(out.empty()) transfer(); return out.top(); }  
    bool empty() { return in.empty() && out.empty(); }  
};  
Time: push O(1), pop O(1) amortized
```

--- TREES (NOT in your PDFs) ---

Q. Diameter of a binary tree (longest path between any two nodes).

```
int ans = 0;  
int depth(TreeNode* root) {  
    if(!root) return 0;  
    int left = depth(root->left);  
    int right = depth(root->right);  
    ans = max(ans, left + right);  
    return 1 + max(left, right);  
}  
int diameterOfBinaryTree(TreeNode* root) { depth(root); return ans; }  
Time: O(n)
```

Q. Check if two binary trees are identical.

```
bool isSameTree(TreeNode* p, TreeNode* q) {  
    if(!p && !q) return true;  
    if(!p || !q) return false;  
    return p->val==q->val && isSameTree(p->left,q->left) && isSameTree(p->right,q->right);  
}
```

```

511
512 Q. Mirror/Invert a binary tree.
513     TreeNode* invertTree(TreeNode* root) {
514         if(!root) return nullptr;
515         swap(root->left, root->right);
516         invertTree(root->left);
517         invertTree(root->right);
518         return root;
519     }
520
521 Q. Path sum – does any root-to-leaf path sum to target?
522     bool hasPathSum(TreeNode* root, int target) {
523         if(!root) return false;
524         if(!root->left && !root->right) return target == root->val;
525         return hasPathSum(root->left, target-root->val) ||
526             hasPathSum(root->right, target-root->val);
527     }
528
529 --- VECTORS / ARRAYS (NOT in your PDFs) ---
530
531 Q. Find the intersection of two arrays (unique elements).
532     vector<int> intersection(vector<int>& a, vector<int>& b) {
533         unordered_set<int> s(a.begin(), a.end());
534         vector<int> res;
535         for(int x : b)
536             if(s.count(x)) { res.push_back(x); s.erase(x); }
537         return res;
538     }
539     Time: O(n+m)
540
541 Q. Three sum – find all triplets that sum to zero.
542     vector<vector<int>> threeSum(vector<int>& nums) {
543         sort(nums.begin(), nums.end());
544         vector<vector<int>> res;
545         for(int i=0; i<nums.size()-2; i++) {
546             if(i>0 && nums[i]==nums[i-1]) continue; // skip duplicates
547             int lo=i+1, hi=nums.size()-1;
548             while(lo<hi) {
549                 int sum=nums[i]+nums[lo]+nums[hi];
550                 if(sum==0) { res.push_back({nums[i],nums[lo],nums[hi]}); lo++; hi--; while(lo<hi&&nums
[lo]==nums[lo-1])lo++; }
551                 else if(sum<0) lo++;
552                 else hi--;
553             }
554         }
555         return res;
556     }
557     Time: O(n²)
558
559 Q. Longest consecutive sequence in unsorted array.
560     int longestConsecutive(vector<int>& nums) {

```

```

561     unordered_set<int> s(nums.begin(), nums.end());
562     int best=0;
563     for(int n : s) {
564         if(!s.count(n-1)) { // start of a sequence
565             int cur=n, len=1;
566             while(s.count(cur+1)) { cur++; len++; }
567             best=max(best,len);
568         }
569     }
570     return best;
571 }
572 Time: O(n)
573
574 Q. Find the majority element (appears more than n/2 times).
575 int majorityElement(vector<int>& nums) {
576     int candidate=nums[0], count=1;
577     for(int i=1; i<nums.size(); i++) {
578         if(count==0) { candidate=nums[i]; count=1; }
579         else if(nums[i]==candidate) count++;
580         else count--;
581     }
582     return candidate;
583 }
584 // Boyer-Moore Voting Algorithm. Time O(n), Space O(1).
585
586 --- DYNAMIC PROGRAMMING (simple ones likely asked) ---
587
588 Q. Climbing stairs – n stairs, can climb 1 or 2 at a time. Ways?
589 int climbStairs(int n) {
590     if(n<=2) return n;
591     int a=1, b=2;
592     for(int i=3; i<=n; i++) { int c=a+b; a=b; b=c; }
593     return b;
594 }
595 // Same as Fibonacci. Time O(n), Space O(1).
596
597 Q. Longest Increasing Subsequence (LIS).
598 int lengthOfLIS(vector<int>& nums) {
599     int n=nums.size();
600     vector<int> dp(n,1);
601     int res=1;
602     for(int i=1; i<n; i++) {
603         for(int j=0; j<i; j++)
604             if(nums[j]<nums[i]) dp[i]=max(dp[i], dp[j]+1);
605         res=max(res, dp[i]);
606     }
607     return res;
608 }
609 Time: O(n2), Space: O(n)
610
611

```

=====

ROUND 4 – HR / BEHAVIORAL

=====

Q. Why are you leaving MicroWorld after 3 years?

ANSWER: "I've had a great run at MicroWorld – 16 production modules, kernel programming on both Linux and macOS. But I've reached a point where I want to work in a product team with more cross-functional exposure – code reviews, design discussions, and new platform challenges like Windows. Network Marvels offers that, plus the product scope aligns with my security background."

[Never say: money, politics, no growth – even if true]

Q. Describe a bug that was hard to find.

ANSWER: "In my ransomware module, the SIGKILL was being sent to the correct PID – but occasionally a child process was spawning before the kill landed, which continued the bulk renames. I used dmesg tracing to see the timeline, realized I needed to kill the entire process group, not just the PID. Changed kill() to killpg() – fixed. The lesson: in kernel code, always think about process hierarchy."

Q. How do you learn a new technology quickly?

ANSWER: "I've done it with every major component I've built – I had never used fanotify before MW Guardian, or libsodium before Ed25519 signing. My approach: read the man page and primary docs first (not tutorials), write the simplest possible working prototype, then iterate. I estimate 4-5 weeks to be productive on Windows KMDF given my Linux kernel background."

Q. Have you ever disagreed with a technical decision?

ANSWER: "When we were designing the FIM module, the initial plan was to poll file hashes every 5 minutes. I proposed using FSEvents instead – it's event-driven, no polling overhead, and catches changes the instant they happen. I showed a prototype with latency numbers. The team agreed and we went with FSEvents. I think the key is to back disagreements with data, not just opinion."

Q. Where do you see yourself in 2-3 years?

ANSWER: "I want to be productive across the full platform stack – Linux, macOS, and Windows kernel. In 2-3 years I'd like to be leading the design of security modules end-to-end – from the kernel driver to the user-facing policy engine. I'm also interested in threat analysis and feeding real attack patterns back into the product."

Q. What is your notice period?

Know this exactly. State it proactively and confidently.

Q. Why should we hire you over other candidates?

ANSWER: "I bring production kernel-level experience that most candidates at this level don't have. I haven't just read about fanotify or ftrace – I've used them in production security products

used by enterprise customers. My ramp-up on Windows will be fast
because the fundamentals – process model, IPC, interrupt handling –
are the same. I ship things."

2 SMART QUESTIONS TO ASK THE INTERVIEWER

(Pick whichever fits the conversation)

QUESTION 1 – Technical Direction:

"I noticed the JD mentions Windows SDK and KMDF. Are these used for
new feature development, or primarily for maintaining existing drivers?
I ask because I want to understand whether there's design ownership
involved or primarily implementation work."

WHY THIS WORKS: Shows you think about ownership, not just task
execution. Signals that you want to contribute at design level –
consistent with your resume.

QUESTION 2 – Team and Stack:

"What does the code review and design review process look like on
the team? For example, when a new module is being designed, how
many people are involved and at what stage does the architect sign off?"

WHY THIS WORKS: Shows you understand professional software development
lifecycle (exactly what the JD asks for – peer reviews, LLD, etc.).
Shows maturity. Any good manager will appreciate this question.

BACKUP QUESTION 3 – Product scope:

"Is the team's security product focused on the enterprise EDR space,
or are there plans to expand into consumer or cloud-native
endpoint monitoring?"

WHY THIS WORKS: Shows business awareness, not just coding interest.

SALARY NEGOTIATION ADDENDUM

When asked "What is your expected CTC?":

"My current CTC is 5 LPA. Given my specialised background – kernel
modules, binary integrity systems, 16 production security modules –
I'm looking at 8 to 9 LPA. I understand there's a Windows ramp-up
involved, which I've factored in. But the core systems depth is
immediately deployable."

If they say "We can offer 6-6.5 LPA":

"I appreciate the offer. 6.5 is below my target for this skill set.
My bottom is 8. Is there room to move, perhaps with a 6-month review

714 clause tied to the Windows ramp-up?"
715
716 If they ask "Why 8 when you only have 3 years?":
717 "The years matter less than the depth. Writing production kernel modules
718 and cryptographic signing pipelines is not typical 3-year experience.
719 I'm not comparing to general software engineers – I'm comparing to
720 kernel/security engineers, where this is competitive."

721
722
723 =====
724 LAST-MINUTE READ – 30 MINUTES BEFORE 2 PM
725 =====

726
727 static_cast → compile time, fast
728 dynamic_cast → runtime RTTI check, slower
729 placement new → construct in pre-allocated memory
730 emplace_back → construct in-place, no temp
731 std::forward → perfect forwarding in templates
732
733 FAN_OPEN_PERM → blocks exec, waits for ALLOW/DENY
734 FAN_CLOSE_WRITE → notify after write, no blocking
735 spinlock_t → use in interrupt/kernel context (cannot sleep)
736 mutex_t → use in process context (can sleep)
737
738 fork() + COW → pages shared until write
739 mmap → file in address space, no copy
740 inode → metadata, not filename
741 Hard link → same inode, different name
742
743 Merge two sorted lists → dummy node trick, $O(n)$
744 Remove Nth from end → two pointers $n+1$ apart, $O(n)$
745 Palindrome check list → reverse second half, $O(n)$
746 Next greater element → monotonic stack, $O(n)$
747 MinStack getMin $O(1)$ → auxiliary min stack
748 Majority element → Boyer-Moore voting, $O(n)$
749 Longest consecutive → hash set, check $n-1$, $O(n)$
750 Climb stairs → Fibonacci pattern, $O(n)$

751
752 =====
753 ALL THE BEST – TOMORROW 2 PM. YOU HAVE THE EXPERIENCE.
754 LINK EVERY ANSWER BACK TO WHAT YOU ACTUALLY BUILT.
755 THAT IS YOUR EDGE.
756 =====

757